

WEB APPLICATION PENETRATION TEST REPORT

Target: Metasploitable 2 — DVWA

Lab IP: 192.168.56.102

Prepared by: ApexPlanet Internship Candidate

Date: March 31, 2026

Program: ApexPlanet Security Internship — Capstone Project

1.ABOUT THIS REPORT

the work I did during Days 49 to 60 of my ApexPlanet security internship. The goal was to simulate a real penetration test — against a deliberately vulnerable machine called Metasploitable 2. In simple terms, I set up a fake "broken" server on my own laptop inside VirtualBox, then used real hacking tools to attack it, documented every step, and finally put on my defender hat to figure out how the attack could have been detected and stopped. Everything happened in an isolated network with no internet access, so nothing outside my laptop was touched. The target was running DVWA — Damn Vulnerable Web App — which is a website intentionally built with security holes for exactly this kind of practice. Think of it as a punching bag designed to be hit.

Item	Detail
My machine (Kali Linux)	192.168.56.103 — the attacker
Target machine	192.168.56.102 — Metasploitable 2 running DVWA
Network	Host-only, completely isolated, no internet
Duration	Days 49 to 60, roughly 12 days
Methodology	OWASP Testing Guide v4 + standard pentest phases
Overall risk found	CRITICAL — multiple ways to fully own the server

2. Lab Setup and Tools

Before doing any attacking I had to build the lab. I used VirtualBox to run three virtual machines on my laptop — Kali Linux as my attacker machine, Metasploitable 2 as the target. All two were connected through a private host-only network so they could talk to each other but had zero internet access.

Getting the IPs sorted was a bit of a journey. The VMs were picking up DHCP addresses instead of the static ones I planned. Eventually Kali settled on 192.168.56.103 and Metasploitable 2 on 192.168.56.102. Once I could ping the target from Kali and see DVWA's login page in the browser, I knew the lab was ready.

Tools I used

Tool	What I used it for
Nmap 7.95	Finding open ports and services on the target
Nikto 2.5.0	Scanning the web server for known vulnerabilities
SQLMap 1.9.11	Automating the SQL injection attack
Burp Suite	Intercepting web traffic and testing manually
Metasploit	Running the vsftpd backdoor exploit
Wireshark 4.6.0	Watching network packets during attacks
Python 3	Writing helper scripts to save evidence

3. Reconnaissance — Scoping Out the Target

The first real step was reconnaissance — basically walking around the outside of the building before trying to break in. I ran two scans here: Nmap to find all the open doors (ports), and Nikto to check the web server for known weaknesses.

Nmap scan results

I ran `nmap -sV -sC` against 192.168.56.102 and honestly the results were shocking even for a deliberately vulnerable machine. It came back with 23 open ports. A normal, hardened server would have maybe 2 or 3.

Here are the ones that caught my eye immediately:

Port	Service	Version	Why it matters
21	FTP	vsftpd 2.3.4	Has a known backdoor — gives a shell in seconds
80	HTTP	Apache 2.2.8	DVWA running here — all web attacks live here
445	Samba	Samba 3.0.20	Famous remote code execution vulnerability
1524	Shell	Bindshell	Someone literally left a root shell open
3306	MySQL	5.0.51a	Database exposed to the network, no firewall
6667	IRC	UnrealIRCd	Another backdoor in this version
8180	HTTP	Apache Tomcat 5.5	Old Tomcat with default admin credentials

Nikto web scan results

Nikto scanned port 80 and found 27 issues. The three that I thought were most serious:

- `phpinfo.php` was sitting publicly accessible. This file dumps out the entire server configuration — PHP version, file paths, installed modules, everything. No attacker should ever be able to see this.
- `phpMyAdmin` was open with no access restrictions. This is the control panel for the MySQL database.

Leaving it public is like leaving your bank's vault door open with a sign saying 'come in'.

- Apache 2.2.8 and PHP 5.2.4 are both ancient versions from around 2008. They have dozens of publicly known vulnerabilities and haven't received security patches in years.

4. Exploitation — Actually

This phase is where I actually tried to exploit the vulnerabilities I found. I did four attacks in total. Each one was done manually first so I understood what was happening, then with tools to show how automated the attacks can be in real life.

Attack 1 — SQL Injection

Severity: CRITICAL | CVSS Score: 9.8 | Location:
/dvwa/vulnerabilities/sqli/

SQL Injection is probably the most classic web vulnerability out there. The DVWA login and user search pages were passing whatever I typed straight into the database query without checking it first. That meant I could type database commands instead of a normal user ID and the server would just run them.

I started manually to understand what was happening. I typed this into the User ID box:

```
' UNION SELECT null, database() -- -
```

The page came back with 'Surname: dvwa' — the database just told me its own name. That confirmed the injection was working. From there I ran SQLMap to automate the rest.

SQLMap found four different injection techniques all working at the same time, then dumped seven entire databases. Most importantly it extracted the users table and automatically cracked all five password hashes:

Username	Stored Hash (MD5)	Actual Password
admin	5f4dcc3b5aa765d61d8327deb882cf99	password
gordonb	e99a18c428cb38d5f260853678922e03	abc123
1337	8d3533d75ae2c3966d7e0d4fcc69216b	charley
pablo	0d107d09f5bbe40cade3de5c71e9e9b7	letmein
smithy	5f4dcc3b5aa765d61d8327deb882cf99	password

Note: MD5 is a hashing algorithm from 1991. It was never meant for passwords and should not be used. All five hashes cracked instantly because they are in publicly available cracking dictionaries.

How to fix it

- Use prepared statements in PHP — this completely prevents SQL injection
- Never build SQL queries by gluing user input into a string
- Store passwords using bcrypt or Argon2, never plain MD5

Attack 2 — Cross-Site Scripting (XSS)

Severity: HIGH | CVSS Score: 7.2 | Location: /dvwa/vulnerabilities/xss_r/

XSS happens when a website takes what you type and puts it directly onto the page without cleaning it. If you type JavaScript instead of your name, the browser runs that JavaScript. In a real attack this is used to steal session cookies, redirect users to fake login pages, or take over accounts silently.

I typed this into the name field on the XSS reflected page:

```
alert('HACKED&nbsp;BY&nbsp;ME')
```

A popup appeared in the browser immediately. That popup proves my JavaScript ran inside the page. In a real scenario this would be a cookie stealer, not a harmless alert.

How to fix it

- HTML encode all user input before putting it on the page
- Add a Content Security Policy header to block inline scripts

Attack 3 — OS Command Injection

Severity: CRITICAL | CVSS Score: 10.0 | Location: /dvwa/vulnerabilities/cmd/

DVWA has a ping utility where you type an IP and it pings it. The problem is the code takes your input and passes it directly to the operating system. So I can add extra commands after the IP using && which means 'do this AND also do this'.

I typed this into the ping box:

```
127.0.0.1 && id && hostname
```

The server came back with:

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)  
metasploitable
```

The server executed my commands and printed the results. This means I could run anything on this server — read files, create users, download malware, anything. This is about as bad as it gets for a web vulnerability.

How to fix it

- Never pass user input to shell commands
- Use PHP's built-in network functions instead of calling system ping
- Whitelist allowed characters — an IP address only needs digits and dots

Attack 4 — vsftpd 2.3.4 Backdoor via Metasploit

Severity: CRITICAL | CVSS Score: 10.0 | Port: 21 (FTP)

This one was wild. Back in 2011 someone secretly added a backdoor to the vsftpd FTP server source code. If you put a smiley face :) in the username during login, the server opens a hidden root shell on port 6200.

Metasploit has a module for this that does it all automatically in about 30 seconds.

```
msf6 > use  
exploit/unix/ftp/vsftpd_234_backdoor msf6 >  
set RHOSTS 192.168.56.102 msf6 > run [*]  
Command shell session opened whoami root
```

Root. Full control of the machine in three commands. No password, no brute force, just a known backdoor in old software. This is why patching matters.

Note: This attack was also demonstrated in Task 4 of the internship. Screenshots are in the GitHub repo.

How to fix it

- Replace vsftpd 2.3.4 immediately — this version should never be used
- Use SFTP instead of FTP — it is encrypted and does not have this issue

5. Incident Response — Switching to the Defender Side

After finishing the attacks I switched sides. Instead of thinking like an attacker, I thought like a security analyst who just found out their server was compromised. The question was: if this were a real company, what would I do right now?

Step 1 — Detection: finding the attack in the logs

Every request made to the web server gets written to Apache's access log at `/var/log/apache2/access.log`. I checked those logs on the Metasploitable VM and could see my own attacks staring back at me. Here is what they looked like:

```
192.168.56.103 - - [28/Mar/2026] GET /dvwa/vulnerabilities/sql?id=1'+OR+'1'=1
192.168.56.103 - - [28/Mar/2026] GET /dvwa/vulnerabilities/cmd?ip=127.0.0.1+&&+id
192.168.56.103 - - [28/Mar/2026] GET /dvwa/vulnerabilities/xss_r/?name=[script>alert(1)[/script]
```

The attacker IP 192.168.56.103 shows up on every line. The URLs contain the exact payloads used — you can see the SQL OR clause, the `&&` command separator, and the script tag all sitting there in the logs. In a real company with a SIEM system, these patterns would trigger automatic alerts.

Step 2 — Containment: stopping the bleeding

Containment means stopping the attack from doing more damage without destroying evidence. The first priority is not to panic and delete everything — logs and memory contain forensic evidence that will be needed later.

- Block the attacker IP immediately at the firewall level
- Take the vulnerable application offline — `service apache2 stop`
- Snapshot the machine's current state before changing anything
- Preserve all log files — copy them somewhere safe before they rotate
- Force password resets on all accounts that were in the leaked database

Step 3 — Eradication: fixing the actual problems

Containment stops the immediate bleeding but the root causes still exist. Eradication means actually fixing the vulnerabilities so the same attack cannot happen again.

- Rewrite all database queries using prepared statements to fix SQL injection

- Add HTML encoding to all output fields to fix XSS
- Remove shell command calls and replace with built-in functions to fix command injection
- Replace vsftpd 2.3.4 with the latest version
- Remove phpinfo.php and add authentication to phpMyAdmin
- Update Apache and PHP to current supported versions

What I learned from playing both sides

The most interesting thing about doing both the attack and the defence is seeing how obvious the attack looks in the logs after the fact. Every single malicious request is right there. The problem is that nobody was watching. A real company would need an automated system — a SIEM or an IDS — to catch these patterns in real time rather than finding them hours later when the damage is done.

6. Summary of Findings

Here is everything I found, ranked by how serious it is. Critical means fix it today. High means fix it this week.

Vulnerability	Severity	CVSS	Fix Priority
SQL Injection	CRITICAL	9.8	Fix today — use prepared statements
OS Command Injection	CRITICAL	10.0	Fix today — remove shell calls
vsftpd 2.3.4 Backdoor	CRITICAL	10.0	Fix today — replace the software
Root shell on port 1524	CRITICAL	10.0	Fix today — close this port immediately
Reflected XSS	HIGH	7.2	Fix this week — encode all output
Outdated Apache 2.2.8	HIGH	7.5	Fix this week — update Apache
phpMyAdmin exposed	HIGH	7.5	Fix this week — add authentication
phpinfo.php accessible	MEDIUM	5.3	Fix this month — delete the file
Directory indexing on	MEDIUM	4.3	Fix this month — disable in config
Missing security headers	LOW	3.1	Fix when possible — add CSP headers

7. Final Thoughts

This project taught me more than any video or course could have. Doing the actual attack — typing a payload into a box and watching the database dump its contents — makes the vulnerability real in a way that just reading about it never does.

The thing that surprised me most was how fast it all was. The SQL injection took under a minute manually and about 3 min with SQLMap. The Metasploit backdoor exploit was three commands. The command injection was one line typed into a form. These are not slow, complicated attacks that take hours. They are fast, scripted, and completely automated. That is why patching and secure coding matter so much.

Playing the defender side also changed how I think about security. When I looked at the logs I could see every attack clearly — but only after it happened. Real security teams need tools that catch these patterns live, alert someone, and block the attacker automatically. That is what a SIEM does, and building one would be a great next step.

Deliverable	Status
Nmap scan	Saved to recon/nmap-scan.txt
Nikto scan	Saved to recon/nikto-scan.txt
SQLMap database dump	Saved to exploitation/SqlInjection/
Cracked passwords CSV	Saved to exploitation/SqlInjection/cracked-passwords.csv
XSS findings	Saved to exploitation/xss/findings.txt
Command injection proof	Saved to exploitation/command-injection/findings.txt
Metasploit findings	Saved to exploitation/metasploit/findings.txt
Post incident report	Saved to incident-response/post-incident-report.md
This PDF report	ApexPlanet_Pentest_Report.pdf

All testing was conducted in an isolated virtual lab environment for educational purposes only. No real systems, networks, or people were targeted or affected at any point.